

Big Data:

Big data is defined into 3 v's volume, variety, velocity

Big data is sets that are too large and complex to manipulate with standard methods

Handle big data:

- hadoop
- spark
- cassandra

Spark:

Apache spark is a general-purpose cluster in memory computing system

It is used for fast data analysis

It abstracts API's in java, scala, and python, and provides an optimized engine that supports general execution graph

Hadoop:

Apache Hadoop is a framework that allows for the distributed processing of large data sets across cluster of cluster of commodity computers using a single programming model

Framework written in java

Designed to solve problem that involve analyzing large data programming model based on google map reduce.

Hadoop core components:

HDFS - Hadoop Distributed File System (storage)

MapReduce (processing)

Main components of HDFS:

Name node
Data node
Secondary Name node

Name node -> (metadata(Name,replicas))
Secondary Name node -> (Backup of MetaData)

Name Node:

Manages the file system namespace and regulates access to file by clients

Metadata:

The entire metadata of files are maintained in memory

Metadata types:

- List of files
- List of blocks for each file
- List of data nodes for each block
- File attribute, eg. creation time, replication factor

A transaction log:

Record creation and deletion of files

Data Node:

One per node in the cluster, manage storage attached to the node that they can run on

A block Server

- Stores data in the local file system

- Stores metadata of a block
- Servers data and meta data to the clients
- Block reports
- Sends a report of all existing block to namenode

Secondary Namenode:

- Renamed as CheckNode in 0.21
- Periodically walks up and takes a checkpoint
- Memory requirements are the same as namenode(big)
- Typically should be on a separate machine in large cluster(>10)
- Directory structure is same as namenode except that it keep a previous checkpoint version in addition to the current
- Helps in recovering a failed namenode

Usage of HDFS:

- Scaling Wide
- Scaling Deep
- Cost effective
- Flexible
- Fault tolerant
- Data block rebalancing

Master-Worker Architecture :

Master->Name Node

Worker->Data Node

Why splitting the data: parallel processing

How many blocks:128 mb

DataNode and NameNode Communication:

- DataNode sends heartbeats for every 3 second via TCP handshake
- Every 10th heart beat is a block report
- NameNode builds metadata from block report

- Communication happens through the port defined for NameNode

NameNode MetaData:

- Meta-data in Memory:
 - The entire metadata is in memory
 - No demand paging of fs meta-data
- Types of Metadata
 - List of files
 - List of blocks for each file
 - List of DataNode for each block
 - File attribute, e.g. access time, replication factor
- A Transaction Log
 - Records file creations, file deletions, etc

Assignment:

Let's say you have 80 TB of data to store and to run MapReduce on this amount of data.

Configuration of datanodes:

8 GB RAM

100 MB/s read-write speed

Total no of nodes=20

Let's assume the replication factor is 4 and block size is 64 mb.

By simple,

=Total amount of data * Replication Factor /Total no of nodes

= $80 \times 4 / 20$

=16 (disk size per datanode)

Reading 80TB data at a speed of 100mb/s using a one node only

=Total data/Read-write speed

= $80 \times 1024 \times 1024 / 100$

=838860.8 seconds

=13981.01 hours

with 20 data node you will be able to finish this job in

= $13981.01 / 20$

=699.05 hours

MapReduce:

Open jar file :
hadoop jar file

hadoop jar <jar-file> <class> <source> <destination>

MapReduce is a distributed programming model used for processing large datasets across clusters of computer. It is part of the Apache Hadoop ecosystem and enables parallel data processing

Instead of processing data on a single machine, hadoop splits data across multiple nodes and processes them in parallel

Handles big data (TBs/PBs)
Parallel Processing
Fault tolerant
Works with commodity hardware

Counting how many times each word appears in a large document dataset

How to make backup of datanode:
replication factor - (default) 3 - makes three copies on different data node
Increase latency

Map Phase

Mapper processes input data and converts it into key-value pairs

Steps:
input data is processed by a mapper function
Mapper outputs (key,value) pairs

Example

Input : Big data is powerful

Mapper output:
(Big, 1)

(data,1)
(is,1)
(powerful,1)

Shuffle and Sort:

Shuffle

It automatically performs shuffle after Map phase
Hadoop groups all values with the same key
It communicates to datanode

Example:

(Big,1)
(Big,1)
(data,1)
(data,1)

Grouped output:

Big->[1,1]
Data->[1,1]

Sort

The keys are sorted before sending to reducers

Example:

Big->[1,1]
Data->[1,1]
is->[1,1]

Reduce

The Reducer aggregates values for each key

Example:

Big->[1,1]->2

Data->[1,1]->2

YARN:

YARN (Yet Another Negotiator) is Hadoop's resource management system

Cluster resources

Job scheduling

Application execution

Hadoop -> disk process

spark -> In memory

Batch Job : Process data on Regular interval

Realtime Job : Process data on instant

Batch : Online Analytical processing (OLAP)

Realtime : Online Transaction processing (OLTP)

Why YARN:

Before Hadoop 2.x: MapReduce handles the both processing and resource management

with YARN :

- Resource management is separate
- Supports multiple processing engines

Example :

- MapReduce
- Spark
- Flink
- Tez

YARN Architecture:

Key Components:

Resource Manager

Main cluster manager

- Allocate cluster resources
- Schedule application

Node Manager:

Runs on each worker node

- Monitor resource usage
- Manage containers

Application Master

- Managing application execution
- Communicating with resource manger

YARN Workflow:

- User submits job
- Resource Manager allocates resources
- Application Master starts
- Container execute tasks
- Results are returned

```
[cloudera@quickstart ~]$ cd /usr/lib
[cloudera@quickstart lib]$ ls
[cloudera@quickstart lib]$ cd hadoop-0.20-mapreduce/
[cloudera@quickstart hadoop-0.20-mapreduce]$ hadoop jar hadoop-examples-mr1.jar
[cloudera@quickstart hadoop-0.20-mapreduce]$ hadoop jar hadoop-examples-mr1.jar
wordcount /jeeva/evm /q
[cloudera@quickstart hadoop-0.20-mapreduce]$ hadoop fs -ls /q
[cloudera@quickstart hadoop-0.20-mapreduce]$ hadoop fs -cat /q/part-r-00000
```


